

SmartParking

Documentazione del progetto Testing e Verifica del Software

Francesco Tomasoni 1080980

A.A. 2025/2026

Indice

1	Introduzione e requisiti	3
1.1	Descrizione del sistema	3
1.2	Requisiti funzionali	3
1.3	Requisiti non funzionali	3
1.4	Macchina a stati	4
2	Modello Asmeta	5
2.1	Il modello (<code>asmeta/src/SmartParking.asm</code>)	5
2.1.1	Il modello semplificato per il model checking	6
2.1.2	Simulazione manuale	7
2.2	Scenari Avalla	9
2.3	AsmetaV – validazione con copertura (Vc)	10
2.3.1	Copertura degli scenari scritti a mano	10
2.3.2	Quello che resta scoperto, e perché	11
2.3.3	Scenari generati automaticamente	13
2.3.4	Cosa dice il confronto	13
2.4	Model checking con AsmetaSMV	13
2.4.1	Model Advisor	16
3	Implementazione Java	17
3.1	Struttura del progetto Maven	17
3.2	La macchina a stati	17
3.3	Interfaccia utente (Vaadin)	18
3.4	Test end-to-end con Selenium	20
4	Testing Java	22
4.1	Struttura dei test	22
4.2	Esecuzione e copertura	23
4.3	Un criterio più forte: MC/DC	24
4.4	Combinatorial testing (pairwise)	25
4.5	Mockito	25
4.6	Model Based Testing	26
4.7	Mutation testing con PIT	27
5	JML e verifica statica (ESC)	29
5.1	Invarianti	29
5.2	Costruttori	29
5.3	Metodo <code>assegnaPosto</code>	30

5.4	Metodo <code>liberaPosto</code>	30
5.5	Metodi <code>getter</code>	31
5.6	Verifica dei contratti con OpenJML (ESC)	31
5.6.1	Prima esecuzione: cinque metodi su sei	31
5.6.2	Perché è invalido: lettura del controesempio	32
5.6.3	La correzione e la seconda esecuzione	32
5.7	Verifica dinamica: i contratti a runtime (RAC)	33
6	Ispezione del codice e analisi statica	34
6.1	Rilettura critica con un LLM	34
6.2	Gli altri due filtri, in breve	35
7	Continuous Integration	36

1 Introduzione e requisiti

1.1 Descrizione del sistema

SmartParking è un parcheggio automatico. Ci sono due sbarre, una in ingresso e una in uscita, dei sensori che si accorgono quando arriva un'auto e un totem che legge il tipo di utente che sta passando. I posti sono di due tipi: standard e riservati ai disabili.

Gli utenti sono tre:

- **STD**: l'utente standard, è l'unico che paga;
- **DISABILE**: ha il permesso, non paga e ha la precedenza sui posti riservati;
- **ABBONATO**: ha l'abbonamento mensile e non paga.

Quando arriva un'auto il sistema la rileva, guarda che utente è e controlla se c'è un posto adatto. Se c'è alza la sbarra, se non c'è nega l'accesso.

Il caso un po' più interessante è quello del disabile: lui i posti riservati li prende per primo, ma se sono finiti il sistema non lo respinge: gli dà un posto standard, se ce n'è uno libero. Questo è il *fallback*. Solo se sono finiti anche i posti standard viene negato l'accesso anche a lui.

In uscita paga solo l'utente STD, gli altri due escono e basta.

1.2 Requisiti funzionali

ID	Descrizione
RF1	Rilevare l'arrivo di un veicolo in ingresso (sens_in) e passare da IDLE a CHK_IN.
RF2	Identificare il tipo di utente (STD, DISABILE, ABBONATO) e passare allo stato VER.
RF3	Per STD/ABBONATO, assegnare un posto standard se posti_std > 0 e aprire la sbarra (INGR).
RF4	Per DISABILE, assegnare in via prioritaria un posto disabile se posti_dis > 0.
RF5	Se per un DISABILE i posti disabili sono esauriti, tentare il fallback su posto standard.
RF6	Se nessun posto è disponibile (inclusi i fallback), negare l'accesso (NEG).
RF7	Da NEG, tornare in IDLE solo dopo che il veicolo si è allontanato (auto_via).
RF8	Al transito in ingresso, decrementare il contatore del posto assegnato e tornare in IDLE.
RF9	Rilevare l'arrivo di un veicolo in uscita (sens_out) e passare a CHK_OUT.
RF10	In uscita, per STD richiedere il pagamento (TARIF) prima della sbarra.
RF11	In uscita, per DISABILE/ABBONATO saltare il pagamento e andare direttamente a USC.
RF12	Restare in TARIF finché il pagamento non è confermato.
RF13	Al transito in uscita, incrementare il contatore, azzerare la memoria del posto e tornare in IDLE.
RF14	Mantenere in memoria il tipo di posto occupato dall'ingresso fino all'uscita.

1.3 Requisiti non funzionali

ID	Descrizione
RNF1	Sicurezza dei contatori: <code>posti_std</code> e <code>posti_dis</code> non devono mai essere negativi né superare la capacità iniziale, in ogni stato raggiungibile.
RNF2	Determinismo: a parità di stato e input monitorati, il sistema produce sempre lo stesso stato successivo.
RNF3	Assenza di stati bloccanti: da ogni stato transitorio esiste sempre un cammino che riporta il sistema in IDLE.

Tutti e tre sono stati verificati formalmente, e vale la pena dire con cosa. RNF1 è esattamente il contenuto delle CTLSPEC da 1 a 6 sul modello e delle due invarianti di classe JML su **Parcheggio**: la stessa proprietà dimostrata due volte, una sul modello astratto e una sul codice.

RNF2 merita una precisazione, perché la regola di assegnazione usa un **choose**, che è un costrutto non deterministico. Il determinismo non viene meno: come spiegato in Sezione 2.1, l'insieme fra cui il **choose** sceglie contiene sempre al massimo un elemento, quindi lo stato successivo resta univoco.

RNF3 corrisponde alla CTLSPEC 8, `ag(stato=TARIF implies ef(stato=IDLE))`, che risulta vera. È importante che sia formulato con “esiste un cammino” e non “su tutti i cammini”: la versione forte è la CTLSPEC 10, con **af** al posto di **ef**, ed è *falsa*, perché un utente che non paga mai resta in TARIF per sempre. Il requisito è scritto nella forma debole apposta.

1.4 Macchina a stati

Qui sotto ci sono gli 8 stati del modello e le transizioni che li collegano. Il giro sopra è quello di ingresso, quello sotto l'uscita, e da IDLE si parte sempre.

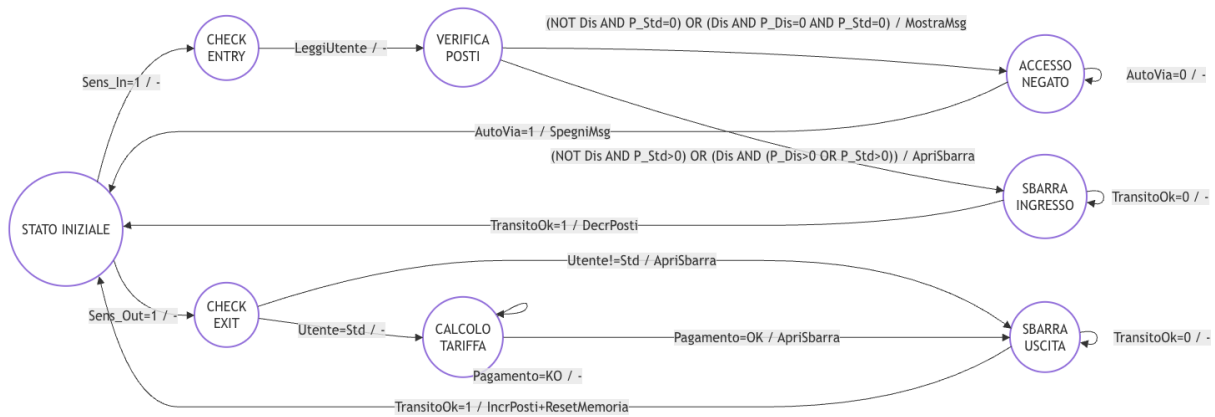


Figura 1: Macchina a stati di SmartParking: gli 8 stati e le transizioni, con il ciclo di ingresso (CHK_IN, VER, INGR oppure NEG) e quello di uscita (CHK_OUT, TARIF, USC).

2 Modello Asmeta

2.1 Il modello (asmata/src/SmartParking.asm)

Partiamo dalla signature. I domini enumerativi sono tre: `StatoSistema` con gli 8 stati, `TipoUtente` con i 3 tipi di utente e `TipoPosto` con i 2 tipi di posto più il valore `NESSUNO`, che vale quando non c'è nessun posto occupato.

Poi ci sono 6 funzioni monitorate, che sono quelle che arrivano dall'esterno: `sens_in` e `sens_out` per i due sensori, `transito_ok` per dire che l'auto è passata sotto la sbarra, `auto_via` per dire che se n'è andata dopo un rifiuto, `utente_rilevato` per il tipo di utente letto dal totem e `pagamento_ok` per l'esito del pagamento.

Le funzioni controllate, cioè la memoria del sistema, sono 4: `stato`, i 2 contatori `posti_std` e `posti_dis`, e `posto_assegnato`, che si ricorda che tipo di posto ha preso l'auto entrata. Quest'ultima serve perché in uscita bisogna sapere quale contatore rimettere a posto.

Le funzioni derivate sono 2. La prima è `assegnabile`, ed è la più interessante delle due perché è binaria: prende un utente e un posto, e dice se quel posto si può dare a quell'utente. Le sue locazioni sono quindi nove, una per ogni combinazione.

Listing 1: Funzione derivata assegnabile

```

1 derived assegnabile: Prod(TipoUtente, TipoPosto) -> Boolean
2
3 function assegnabile($u in TipoUtente, $p in TipoPosto) =
4     ($p = POSTO_STD and posti_std > 0) or
5     ($p = POSTO_DIS and $u = DISABILE and posti_dis > 0)

```

Un posto standard lo do a chiunque, basta che ce ne sia uno libero; un posto disabili lo do solo a un disabili, e solo se ce n'è uno libero. Il vantaggio di averla messa qui è che tutta la politica di assegnazione sta in due righe, in un punto solo.

La seconda derivata è `parcheggioPieno`

Listing 2: Funzione derivata parcheggioPieno (uso di forall)

```

1 function parcheggioPieno =
2     (forall $u in TipoUtente with
3         (not assegnabile($u, POSTO_STD) and not assegnabile($u, POSTO_DIS)))

```

Diventa vera quando *nessun* tipo di utente può più entrare, né su un posto standard né su uno disabili. La controllo nello scenario `test_pieno_totale.avalla`: all'inizio è falsa perché c'è ancora posto, alla fine diventa vera.

C'è infine una funzione statica, che dice quanti posti esistono per ciascun tipo:

Listing 3: Funzione statica unaria capacita

```

1 static capacita: TipoPosto -> Integer
2
3 function capacita($p in TipoPosto) =
4     if $p = NESSUNO then 0 else 1 endif

```

Statica perché il suo valore non dipende dallo stato e non cambia mai durante l'esecuzione: è una proprietà del parcheggio, non della sua evoluzione. La usa l'`init` per fissare i due contatori, `posti_std = capacita(POSTO_STD)` e `posti_dis = capacita(POSTO_DIS)`, invece di ripetere il numero 1 due volte. Il caso `NESSUNO` vale 0 perché non è un posto vero ma il segnaposto per "nessun posto occupato".

Ora passiamo alle regole. Una delle principali è `r_gestione_VER`, dove si decide chi entra e chi no. Ci sono tre costrutti in poche righe: un `let` per non ripetere `utente_rilevato` ogni volta, un `if` sulla derivata n-aria e un `choose` con la sua clausola `ifnone`.

Listing 4: Regola `r_gestione_VER` (assegnazione posto, con `let`, `if`, `choose` e derivata)

```

1 rule r_gestione_VER =
2   let ($u = utente_rilevato) in
3     if assegnabile($u, POSTO_DIS) then
4       par
5         stato := INGR
6         posto_assegnato := POSTO_DIS
7       endpar
8     else
9       choose $p in TipoPosto with assegnabile($u, $p) do
10        par
11          stato := INGR
12          posto_assegnato := $p
13        endpar
14      ifnone
15        stato := NEG
16    endif
17  endlet

```

La priorità è nell'ordine: si prova prima con il posto disable, condizione vera solo se l'utente è disable e c'è un riservato libero. Se quel test fallisce si passa al `choose`, che pesca un posto fra quelli ancora assegnabili all'utente; se non ne trova nessuno scatta l'`ifnone` e il sistema va in NEG. Quel ramo copre due casi diversi in una volta: l'ingresso normale di uno STD o di un abbonato, e il fallback del disable quando i riservati sono finiti.

Sul `choose` vale la pena essere precisi, perché è un costrutto non deterministico e qui il comportamento deve invece restare prevedibile: gli scenari controllano `posto_assegnato` con valori esatti, e una scelta arbitraria li renderebbe insensati. La garanzia viene dalla posizione in cui il `choose` è stato messo. Ci si arriva solo quando `assegnabile($u, POSTO_DIS)` è già risultata falsa, quindi `POSTO_DIS` è fuori dai candidati per costruzione; NESSUNO non è mai assegnabile perché non soddisfa nessuno dei due rami della derivata. Resta al massimo `POSTO_STD`: l'insieme da cui si sceglie ha sempre uno o zero elementi, e nel secondo caso si passa per l'`ifnone`.

Le altre regole sono più semplici. `r_gestione_IDLE` guarda quale dei due sensori si è acceso e va in `CHK_IN` o in `CHK_OUT`. `r_gestione_CHK_IN` passa subito a VER, serve solo a rappresentare il tempo di lettura del totem. `r_gestione_INGR` aspetta il transito e poi scala il contatore giusto, guardando `posto_assegnato`. `r_gestione_CHK_OUT` manda in TARIF solo se l'utente è STD, altrimenti va dritto in USC. `r_gestione_TARIF` resta ferma finché il pagamento non va a buon fine. `r_gestione_USC` rimette a posto il contatore e azzerava `posto_assegnato`.

2.1.1 Il modello semplificato per il model checking

Il modello principale non è utilizzabile per il model checking, quindi ne esiste una copia semplificata in `SmartParking_MC.asm`. Le differenze sono due:

La prima riguarda i costrutti: nella copia non ci sono né le funzioni derivate né il `choose`, e la scelta del posto è riscritta come una catena di `if` "appiattiti". Il comportamento è lo stesso, ma la traduzione verso NuSMV va liscia.

La seconda riguarda i contatori: nel modello principale sono `Integer`, e AsmetaSMV i domini infiniti non li accetta, quindi qui stanno su un dominio finito `Posti = {0,1,2}`. Il 2 è lasciato

apposta anche se non serve: con $\{0,1\}$ le proprietà $\text{ag}(\text{posti_std} \leq 1)$ e $\text{ag}(\text{posti_dis} \leq 1)$ sarebbero state vere per forza, solo per come è fatto il tipo, e non avrebbero verificato niente. Lasciando dentro il 2 il model checker deve davvero dimostrare che quel valore non viene mai raggiunto.

2.1.2 Simulazione manuale

Sono state condotte diverse simulazioni manuali tramite il simulatore, qui ne sono presenti due:

Nello screenshot c'è il giro completo di un ingresso STD: metto **sens_in** a true e passo in **CHK_IN**, poi imposto l'utente e la macchina va in **VER**, da lì in **INGR** perché c'è posto, e infine con **transito_ok** torno in **IDLE** con **posti_std** sceso da 1 a 0.

	Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5
	M	sens_in	false	true	true	true	true	
	M	sens_out	false	false	false	false	false	
	C	stato	IDLE	IDLE	CHK_IN	VER	INGR	IDLE
	M	utente_rilevato				STD	STD	
	C	posti_std				1	1	0
	C	posti_dis				1	1	1
	C	posto_assegnato					POSTO_STD	POSTO_STD
	M	transito_ok					true	

Figura 2: Simulatore – ingresso STD completo

Qui invece c'è l'uscita, sempre nella stessa sessione. Si vede il passaggio per **TARIF**, che è la parte che riguarda solo gli STD, e alla fine **posti_std** torna a 1 e **posto_assegnato** torna a NESSUNO.

Figura 3: Simulatore – uscita STD con pagamento

2.2 Scenari Avalla

Gli scenari sono questi:

Scenario	Cosa prova
test_abbonato.avalla	Giro completo di un ABBONATO: entra, e in uscita salta il pagamento.
test_disabile.avalla	Giro completo di un DISABILE su posto riservato; controllo che <code>posto_assegnato</code> resti <code>POSTO_DIS</code> per tutta la sosta e si azzeri solo in uscita.
test_pieno.avalla	Uno STD prende l'unico posto, un secondo STD viene respinto.
test_disabile_fallback.avalla	Il fallback: due disabili di fila, il secondo trova i riservati finiti e prende un posto standard.
test_pagamento_std.avalla	Uscita STD con un pagamento che va male (resta in <code>TARIF</code>) e poi uno che va bene.
test_pieno_totale.avalla	Parcheggio pieno del tutto (uno STD più un disabile), il terzo utente viene respinto.
test_funzioni_narie.avalla	Valuta direttamente <code>assegnabile</code> e <code>capacita</code> con i loro argomenti, prima e dopo un ingresso, per mostrare che la derivata cambia con lo stato e la statica no.

Sono tutti scritti a mano. I primi sei coprono un comportamento ciascuno ed è su di essi che si misura la copertura nella sezione seguente; il settimo è di natura diversa, perché non prova una sequenza di stati ma valuta direttamente le funzioni n-arie con i loro argomenti, prima e dopo un ingresso, per mostrare che la derivata cambia con lo stato mentre la statica no. Agli scenari generati automaticamente è invece dedicata la §2.3.3.

Nell'animazione dell'abbonato la cosa da guardare è la fase di uscita: da `CHK_OUT` si salta direttamente a `USC`, `TARIF` non compare proprio. È esattamente la differenza tra abbonato e utente STD.

Type	Functions	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13
C	step__	4	5	6	7	8	8	8	8	8	8
C	stato	IDLE	CHK_OUT	USC	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE
C	sens_in	false	false	false	false	false	false	false	false	false	false
C	result	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato	ATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO
C	posto_assegnato	_STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO
C	transito_ok	false	false	true	true	true	true	true	true	true	true
C	posti_std	0	0	0	1	1	1	1	1	1	1
C	sens_out	true	false	false	false	false	false	false	false	false	false

Figura 4: Animazione di test_abbonato.avalla

Qui invece si vedono i due disabili uno dietro l'altro: il primo prende il posto riservato, il secondo lo trova occupato e finisce su quello standard. Tutti i check passano.

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11
C	step__	0	1	2	3	4	5	6	7	8	9	9	9
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE
C	posto_assegnato				POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C	transito_ok				true	false	false	false	true	true	true	true	true
C	postl_dis				0	0	0	0	0	0	0	0	0
C	postl_std								0	0	0	0	0

Figura 5: Animazione di test_disabile_fallback.avalla

E qui il parcheggio saturo, con il terzo utente respinto e `parcheggioPieno` che passa a true.

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C	step__	0	1	2	3	4	5	6	7	8	9	10	11	12	13	13	13
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	CHK_IN	VER	NEG	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		STD	STD	STD	STD	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE
C	posto_assegnato				POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS
C	transito_ok				true	false	false	false	true	false	false	false	false	false	false	false	false
C	postl_std				0	0	0	0	0	0	0	0	0	0	0	0	0
C	postl_dis								0	0	0	0	0	0	0	0	0
C	auto_via											true	true	true	true	true	true

Figura 6: Animazione di test_pieno_totale.avalla

2.3 AsmetaV – validazione con copertura (Vc)

2.3.1 Copertura degli scenari scritti a mano

Gli scenari girano con AsmetaV usando l'opzione che calcola anche la copertura.

Lanciando `test_pieno` da solo si ottiene:

Listing 5: Copertura del solo test_pieno.avalla

```

** covered rules: **
r_gestione_CHK_IN() rule 100.00% branch (nessun ramo)
r_gestione_INGR() rule 71.43% branch 33.33%
r_gestione_VER() rule 70.00% branch 75.00%
r_gestione_IDLE() rule 50.00% branch 25.00%
r_gestione_NEG() rule 100.00% branch 50.00%
r_Main() rule 82.35% branch 81.25%
** NOT covered rules: **
r_gestione_USC() r_gestione_TARIF() r_gestione_CHK_OUT()

```

Tre regole su nove non vengono mai attivate, semplicemente perché in quello scenario nessuno esce dal parcheggio, e nessuna delle altre arriva al 100% di branch.

Mettendo insieme i sei scenari di comportamento le regole vengono coperte tutte: le tre che mancano qui le portano `test_abbonato` e `test_disabile` (uscita, e quindi `CHK_OUT` e `USC`) e `test_pagamento_std` (`TARIF`). Per `r_gestione_VER` si percorrono tutte le strade possibili: il ramo `if` con il posto riservato lo copre `test_disabile`, il `choose` che ripiega sullo standard lo coprono `test_abbonato` e il fallback di `test_disabile_fallback`, la clausola `ifnone` che porta in `NEG` la copre `test_pieno` — e infatti `test_pieno_totale`, che da solo le attraversa tutte e tre, porta `r_gestione_VER` al 100% di branch.

2.3.2 Quello che resta scoperto, e perché

La branch coverage invece non si chiude. I rami che restano fuori sono quelli dei *self-loop*: i casi in cui la guardia di una regola è falsa proprio mentre il sistema si trova in quello stato. In `r_gestione_IDLE`, per esempio, uno dei quattro rami è “nessuno dei due sensori è attivo”, cioè uno step a vuoto: è un comportamento legittimo del modello, ma nessuno scenario ha motivo di provocarlo, perché gli scenari sono scritti per raccontare qualcosa che succede.

È esattamente lo stesso fenomeno che si ripresenta sul codice Java, dove l’unico branch non coperto da JaCoCo è il `default` irraggiungibile dello switch (Sezione 4). In entrambi i casi la copertura mancante non segnala un buco nei test, ma un ramo che esiste per completezza strutturale e che nessuna esecuzione sensata attraversa.

```
** Coverage Info: **
** covered rules: **
SmartParking::r_gestione_USC()
-> branch coverage:      33.33%
-> rule coverage:        75.00%
-> update rule coverage: 75.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_IN()
-> branch coverage:      - (no branches to be covered)
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_INGR()
-> branch coverage:      83.33%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_VER()
-> branch coverage:      100.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_IDLE()
-> branch coverage:      75.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_NEG()
-> branch coverage:      100.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_TARIF()
-> branch coverage:      50.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_Main()
-> branch coverage:      100.00%
-> rule coverage:        100.00%
-> update rule coverage: - (no update rules to be covered)
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_OUT()
-> branch coverage:      100.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
** NOT covered rules: **

validation terminated without errors
validating (w coverage) finished
```

Figura 7: Report di copertura AsmetaV (Vc)

2.3.3 Scenari generati automaticamente

Coprire tutte le regole scrivendo gli scenari a mano è fattibile su un modello di questa dimensione, ma la domanda naturale è se il lavoro si possa far fare a una macchina. Ho provato due strade, e il risultato più interessante non è stato guadagnare copertura ma scoprire quanto poco significhi la parola “copertura” senza specificare il criterio.

La prima è l’export dal simulatore: si fanno un po’ di step con input random e si salva la traccia con “export to avalla”. Un primo tentativo da 15 step copriva solo 6 regole su 9 – restavano fuori `r_gestione_NEG`, `r_gestione_TARIF` e `r_gestione_USC`. Ho raddoppiato a 30 step, ed è nato `test_random_30steps.avalla`: la rule coverage è arrivata al 100%, ma la branch coverage no – `r_gestione_USC` al 33%, `r_gestione_TARIF` al 50%, `r_gestione_IDLE` al 75%.

La seconda strada è ATGT, che genera gli scenari a partire dal modello invece che da una passeggiata casuale. Gli scenari prodotti sono cinque file `testtest*.avalla` e stanno in `asmeta/src/abstractests/`.

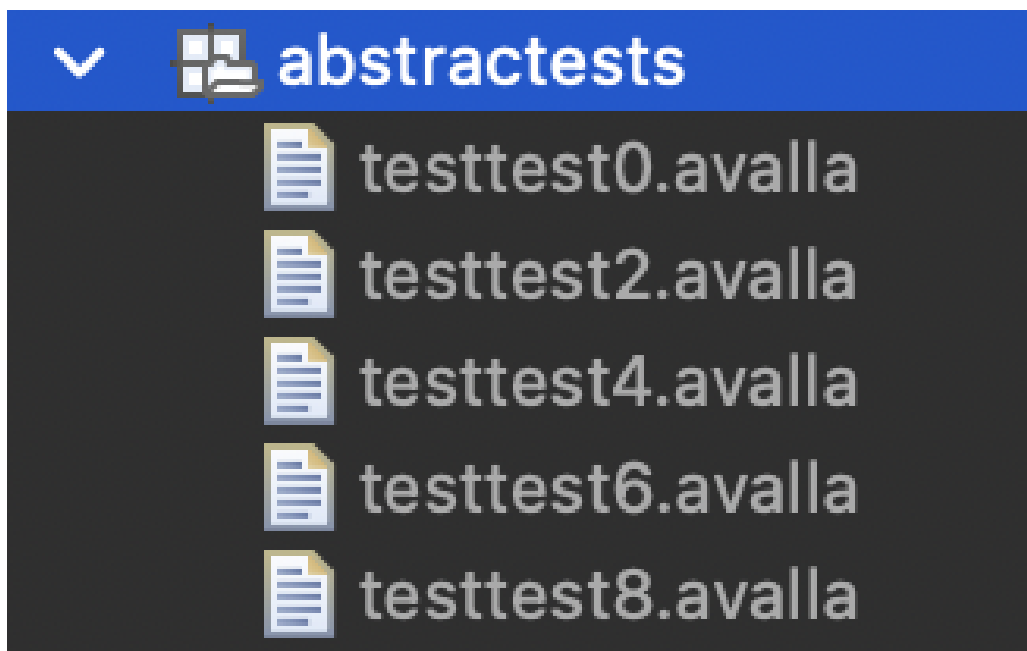


Figura 8: Elenco degli scenari generati da ATGT

2.3.4 Cosa dice il confronto

Il limite è lo stesso per tutte e due le strade, e si capisce guardando un caso concreto. Il ramo del pagamento fallito in `TARIF` vuole che `pagamento_ok` sia false proprio mentre il sistema si trova in `TARIF`: è una combinazione che a caso capita raramente e che va cercata di proposito. Infatti quel ramo lo copre solo lo scenario scritto apposta, `test_pagamento_std`.

La generazione automatica quindi conviene, perché il grosso della copertura lo porta a casa senza fatica, ma non sostituisce gli scenari scritti a mano: quelli restano necessari per i casi limite, che sono poi proprio quelli su cui un sistema del genere tende a rompersi.

2.4 Model checking con AsmetaSMV

Le CTLSPEC sono 10. Otto sono vere e stanno in tre gruppi: safety (cose che non devono mai succedere), raggiungibilità (cose che devono poter succedere) e liveness (cose che prima o poi devono succedere). Le altre due sono false apposta, per vedere il controesempio.

#	CTLSPEC	Categoria	Esito
1	ag(posti_std >= 0)	Safety	TRUE
2	ag(posti_dis >= 0)	Safety	TRUE
3	ag((stato=INGR and posto_- assegnato=POSTO_STD) implies posti_std>0)	Safety	TRUE
4	ag(posti_std <= 1)	Safety	TRUE
5	ag(posti_dis <= 1)	Safety	TRUE
6	ag((stato=INGR and posto_- assegnato=POSTO_DIS) implies posti_dis>0)	Safety	TRUE
7	ef(stato = TARIF)	Raggiungibilità	TRUE
8	ag(stato=TARIF implies ef(stato=IDLE))	Liveness	TRUE
9	ag(stato != NEG)	Falsa (volu- ta)	FALSE
10	ag(stato=TARIF implies af(stato=IDLE))	Falsa (volu- ta)	FALSE

Tabella 4: Le 10 CTLSPEC sottoposte ad AsmetaSMV: 8 verificate, 2 volutamente false.

Le prime sei dicono in sostanza che i contatori non impazziscono: non vanno mai sotto zero, non vanno mai sopra la capacità, e non mando mai un'auto dentro su un tipo di posto che non ha più disponibilità. La settima dice che lo stato di pagamento è raggiungibile, l'ottava che da TARIF si riesce sempre a tornare in IDLE.

Sulle due false vale la pena spendere due parole in più.

La #9 dice "l'accesso non viene mai negato". NuSMV la smonta con un controesempio di 9 stati: entra un abbonato che si prende l'unico posto standard, arriva un secondo veicolo, la verifica non trova posto e va in NEG. La cosa interessante è che in quella traccia `posti_dis` è ancora 1: il controesempio, oltre a dire che NEG è raggiungibile, conferma che il posto disabile non viene dato a chi non ne ha diritto nemmeno quando è l'ultima risorsa rimasta.

La #10 è la stessa cosa della #8 ma con `af` al posto di `ef`. La #8 chiede che da TARIF *esista* un cammino che torna in IDLE, la #10 che ci si torni su *tutti* i cammini. Ed è falsa, perché niente vieta all'ambiente di tenere `pagamento_ok` a false per sempre: un utente che non paga mai. Il controesempio è proprio quello, il loop infinito in TARIF. Sono rimaste tutte e due nel modello perché messe una accanto all'altra fanno vedere bene la differenza tra i due operatori.

```

model checking (w execution) on /Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/eclipse_asmeta_
Execution of NuSMV code ...
-----
> NuSMV -dynamic -coi -quiet SmartParking_MC.smv
-- specification AG posti_std >= 0 is true
-- specification AG posti_dis >= 0 is true
-- specification AG ((posto_assegnato = POSTO_STD & stato = INGR) -> posti_std > 0) is true
-- specification AG posti_std <= 1 is true
-- specification AG posti_dis <= 1 is true
-- specification AG ((stato = INGR & posto_assegnato = POSTO_DIS) -> posti_dis > 0) is true
-- specification EF stato = TARIF is true
-- specification AG (stato = TARIF -> EF stato = IDLE) is true
-- specification AG stato != NEG is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 1.2 <-
  sens_in = true
-> State: 1.3 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.4 <-
  stato = VER
-> State: 1.5 <-
  stato = INGR
  transito_ok = true
  posto_assegnato = POSTO_STD
-> State: 1.6 <-
  stato = IDLE
  transito_ok = false
  sens_in = true
  posti_std = 0
-> State: 1.7 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.8 <-
  stato = VER
-> State: 1.9 <-
  stato = NEG
-- specification AG (stato = TARIF -> AF stato = IDLE) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 2.2 <-
  sens_out = true
-> State: 2.3 <-
  stato = CHK_OUT
  sens_out = false
  utente_rilevato = STD
-- Loop starts here
-> State: 2.4 <-
  stato = TARIF
  utente_rilevato = ABBONATO
-> State: 2.5 <-

model checking (w execution) finished

```

Figura 9: Esito delle 10 CTLSPEC in AsmetaSMV

2.4.1 Model Advisor

Il Model Advisor passa anche lui per la traduzione in NuSMV, quindi sul modello principale non parte proprio: dà “Error Domain Integer not supported”. Girando sul modello semplificato, quello che esce è questo:

- **MP1, MP3, MP4, MP7** non segnalano niente: nessun aggiornamento inconsistente, nessuna assegnazione banale, nessuna locazione da rimuovere o mai aggiornata.
- **MP2** (“conditional rule not complete”) segnala invece tutte le regole di transizione. Sono però falsi positivi: l’advisor segnala ogni **if** senza **else**, ma in ASM l’else che manca vuol dire “non aggiornare niente”, ed è esattamente quello che serve per i self-loop. **r_gestione_NEG** deve restare in NEG finché l’auto non se ne va, e **r_gestione_TARIF** deve restare in TARIF se il pagamento non passa. Con un else esplicito il comportamento sarebbe identico ma il codice più lungo.
- **MP5/MP6** dicono “Domain Posti could be reduced: element {2} could be removed”. Anche questo non è un difetto: il 2 c’è apposta, come spiegato sopra, e l’advisor sta solo confermando che non viene mai raggiunto. È la stessa informazione delle CTLSPEC 4 e 5, per un’altra strada.

```

MP2: Every case rule without otherwise must be complete
NONE

MP3: Choose rule is always/sometimes/never not empty
NONE

MP3: Forall rule is always/sometimes/never not empty
NONE

MP3: Conditional rule eval to true
NONE

MP4: No assignment is always trivial
NONE

MP5: For every domain element e, there exists a location which has value e
Domain Posti could be reduced in size. Elements {2} could be removed.

MP6: Every controlled location can take any value in its codomain
Function posti_dis does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.
Function posti_std does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.

MP7: a location could be removed
NONE

MP7: a controlled location is never updated
NONE

MP7: a controlled location could be static
NONE

model advising finished

```

Figura 10: Output del Model Advisor (estratto)

3 Implementazione Java

3.1 Struttura del progetto Maven

Il progetto Java sta nella cartella `java/` ed è un progetto Maven normale, su Java 17. Le dipendenze principali sono Spring Boot con Vaadin 24 per la UI, JUnit 5 e Mockito per i test, JaCoCo per la copertura.

Il cuore sta nel package `it.tvsw.smartparking.core`, che contiene:

- `StatoSistema`, `TipoUtente`, `TipoPosto`: tre enum, uguali ai domini del modello ASM;
- `Parcheggio`: il nucleo, ed è l'unica classe con i contratti JML (Sezione 5);
- `SensorInput`: mette insieme i sei input monitorati di uno step;
- `Display`: un'interfaccia per le notifiche, che poi serve per Mockito;
- `SmartParkingFSM`: la macchina a stati vera e propria.

Fuori dal core ci sono solo la vista Vaadin e la classe di avvio di Spring Boot.

`Parcheggio` e `SmartParkingFSM` sono dichiarate `final` e tutte le classi del core sono serializzabili: non era così all'inizio, sono due modifiche arrivate dopo l'analisi statica e le spiego in Sezione 6.2.

3.2 La macchina a stati

`SmartParkingFSM` ha un metodo `step(SensorInput)` che fa la stessa cosa della main rule dell'ASM: guarda in che stato siamo e chiama il gestore giusto.

Listing 6: `step()` – dispatch sullo stato corrente

```
1 public void step(SensorInput input) {
2     if (input == null) {
3         throw new IllegalArgumentException("input non puo' essere null");
4     }
5     switch (stato) {
6         case IDLE:      gestisciIdle(input); break;
7         case CHK_IN:    gestisciChkIn(); break;
8         case VER:       gestisciVer(input); break;
9         case NEG:       gestisciNeg(input); break;
10        case INGR:      gestisciIngr(input); break;
11        case CHK_OUT:   gestisciChkOut(input); break;
12        case TARIF:     gestisciTarif(input); break;
13        case USC:       gestisciUsc(input); break;
14        default:
15            throw new IllegalStateException("Stato non gestito: " + stato);
16    }
17 }
```

Ogni `case` corrisponde a una regola dell'ASM, quindi il confronto tra i due livelli si fa a colpo d'occhio. Il ramo `default` non è raggiungibile, dato che i casi coprono tutti e otto i valori dell'enum: è una rete di sicurezza per il giorno in cui qualcuno aggiungesse uno stato nuovo, e sarà l'unico branch che JaCoCo non riuscirà a coprire (Sezione 4). C'è però una differenza che è meglio dire subito. Nel modello ASM il contatore viene scalato solo al transito vero e proprio, cioè in `INGR`, mentre qui la classe `Parcheggio` decide e scala tutto insieme dentro `assegnaPosto`, che viene chiamato da `gestisciVer`. Negli scenari di test del progetto la differenza non si vede, perché nessuno va a controllare lo stato in mezzo ai due passi, ed è una scelta fatta apposta per tenere la classe più semplice. Ne riparlo meglio nell'ispezione del codice, in Sezione 6.

3.3 Interfaccia utente (Vaadin)

La UI è una vista sola, **MainView**, che mostra lo stato corrente e quanti posti sono ancora liberi. I sensori sono simulati con dei bottoni: “Arriva auto STD/DISABILE/ABBONATO”, “Transito”, “Pagamento”, “Auto va via” e “Auto in uscita”. Cliccandoli si fa avanzare la macchina a stati come se arrivassero gli input dai sensori veri.

Questa è la pagina appena aperta: stato IDLE e i due contatori a 1.

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 1

Posti disabili liberi: 1

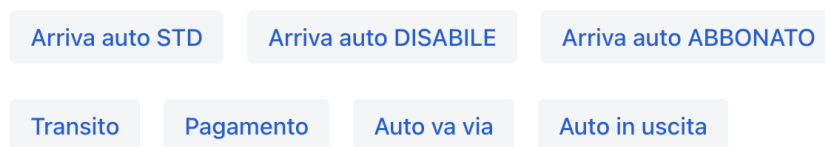


Figura 11: UI Vaadin – stato iniziale

Qui è stato cliccato “Arriva auto STD” e poi “Transito”. Siamo tornati in IDLE ma i posti standard sono scesi a 0.

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 0

Posti disabili liberi: 1

Sistema pronto

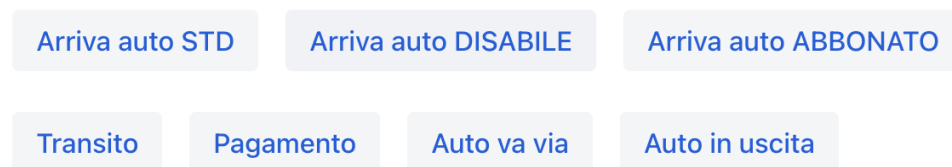


Figura 12: UI Vaadin – dopo un ingresso STD

Con il parcheggio pieno, se arriva un'altra auto compare il messaggio di accesso negato.

SmartParking - Simulatore

Stato: NEG

Posti standard liberi: 0

Posti disabili liberi: 1

Accesso negato

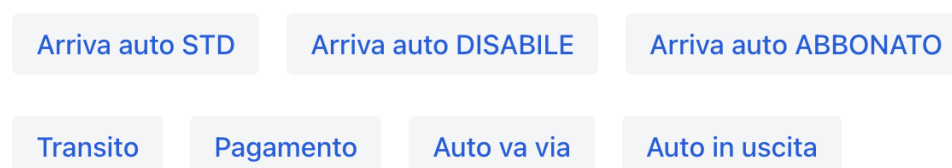


Figura 13: UI Vaadin – accesso negato a parcheggio pieno

E questo è lo stato TARIF: l'utente STD sta uscendo e il sistema si è fermato ad aspettare il pagamento.

SmartParking - Simulatore

Stato: TARIF

Posti standard liberi: 0

Posti disabili liberi: 1

In attesa di pagamento

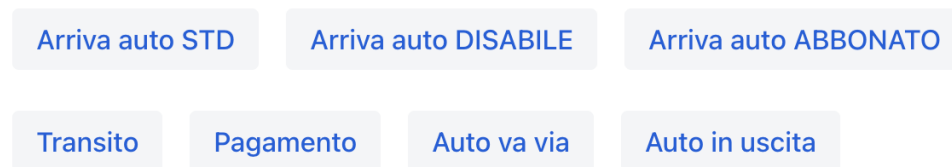


Figura 14: UI Vaadin – attesa pagamento

3.4 Test end-to-end con Selenium

La UI è provata anche con un test end-to-end in Selenium (`UISeleniumTest`).

Quello che fa è: apre la pagina con `ChromeDriver` in modalità headless, aspetta che Vaadin finisca di disegnare la pagina lato client (con `WebDriverWait`, altrimenti trova il DOM vuoto), clicca “Arriva auto STD” e controlla che lo stato mostrato diventi INGR oppure NEG. L’ho fatto partire da Eclipse come un normale test JUnit.

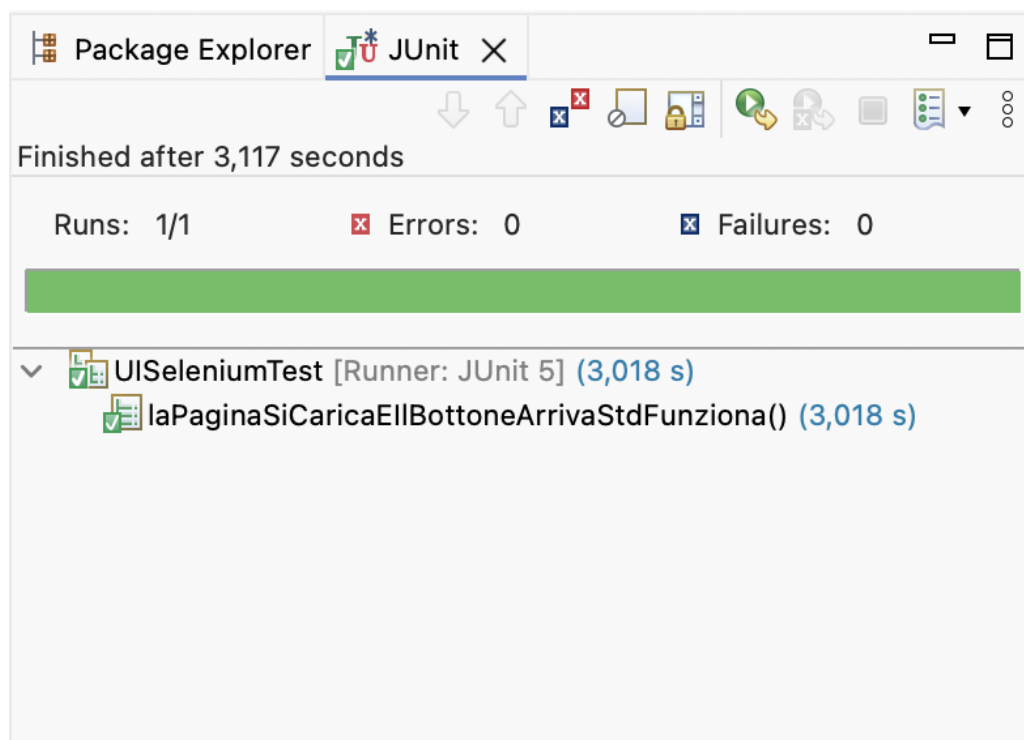


Figura 15: Test Selenium end-to-end superato

4 Testing Java

4.1 Struttura dei test

I test sono divisi in più classi, una per ogni cosa da coprire, invece che tutti insieme. Così se un test si rompe si capisce subito di che tipo di problema si tratta. Le classi sono queste:

Classe	Cosa copre
<code>ParcheggioTest</code>	Costruttori, preconditioni violate, tutti i rami di <code>assegnaPosto</code> e <code>liberaPosto</code> , più un test parametrico con CSV.
<code>SmartParkingFSMTest</code>	Tutti gli 8 stati e le transizioni, il fallback del disabile, il parcheggio pieno, i self-loop di NEG, INGR, TARIF e USC.
<code>McdcVerificaTest</code>	Copertura MC/DC delle due decisioni composte di <code>assegnaPosto</code> .
<code>DisplayMockTest</code>	Le notifiche al <code>Display</code> , controllate con Mockito.
<code>ScenarioAvallaTest</code>	Traduzione di <code>test_pieno.avalla</code> in JUnit (model based testing).
<code>CombinatorialTest</code>	La tabella pairwise a 6 righe.
<code>UISeleniumTest</code>	Smoke test della UI, taggato <code>ui</code> e fuori dalla build normale.

Le asserzioni usate sono di tre famiglie, ciascuna dove serve: `assertEquals` per lo stato e i contatori dopo un'operazione, `assertThrows` per le preconditioni violate (un utente nullo, un contatore già al massimo), e i `verify` di Mockito per i messaggi mandati al display, che non lasciano traccia nello stato e quindi con un `assertEquals` non si controllerebbero.

Due classi usano test **parametrici**: `ParcheggioTest` ha un `@ParameterizedTest` con `@CsvSource` a otto righe che prova tutte le combinazioni utente/contatori di `assegnaPosto`, e `CombinatorialTest` esegue allo stesso modo le sei righe della tabella pairwise. Il vantaggio è che aggiungere un caso costa una riga di CSV invece di un metodo intero.

Come è organizzata questa sezione

Le sottosezioni che seguono sono in ordine di forza crescente, e ognuna esiste perché la precedente non basta. Si parte dalla copertura di statement e branch, che dice quali righe sono state eseguite. Si passa a un criterio più fine, MC/DC, perché eseguire una decisione composta non vuol dire aver provato il contributo di ogni sua condizione. Si arriva infine al mutation testing, che risponde alla domanda che nessuno dei due criteri precedenti può affrontare: quelle righe eseguite, qualcuno ne sta davvero controllando il risultato?

4.2 Esecuzione e copertura



Figura 16: Esecuzione di tutta la suite JUnit

Tutti i test del package `core` passano: sono 59, più quello Selenium che sta fuori dalla build normale.

La copertura si misura con JaCoCo lanciando `mvn verify` e guardando il report. Dal conteggio sono escluse due cose: la UI Vaadin, provata end-to-end con Selenium e che JaCoCo non riesce a misurare, e la classe di avvio di Spring Boot, che sono tre righe di boilerplate. L'esclusione è scritta nel `pom.xml`.

Alla prima misurazione il package `core` stava al 98% di instruction e al 95% di branch. `Parcheggio` era già al 100% su tutte e due, il che tornava con il fatto che i suoi contratti erano già stati dimostrati con ESC. In `SmartParkingFSM` invece restavano fuori tre branch, e vale la pena guardarli uno per uno perché non sono tutti uguali:

- il self-loop di `gestisciUsc`, cioè il caso in cui in USC il transito non è ancora avvenuto;
- il ramo `display == null` di `cambiaStato`;
- il `default` dello switch di `step`.

I primi due erano buchi veri, casi che mi ero semplicemente dimenticato di provare, e li ho chiusi con due test in più (`uscRestaInUscFinoAlTransito` e `fsmFunzionaSenzaDisplay`). Il terzo invece non si può coprire: lo switch gestisce tutti e 8 i valori dell'enum, quindi in quel `default` non ci si arriva mai. L'ho lasciato lo stesso perché lancia un'eccezione ed è una rete di sicurezza se un giorno qualcuno aggiunge uno stato nuovo. È lo stesso tipo di ramo infattibile già incontrato con `AsmetaV` sul modello (§2.3).

Dopo i due test aggiunti, quel `default` è rimasto l'unico branch scoperto di tutto il package.

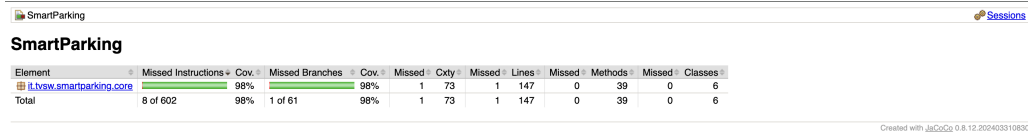


Figura 17: Report JaCoCo – totale (solo package core)

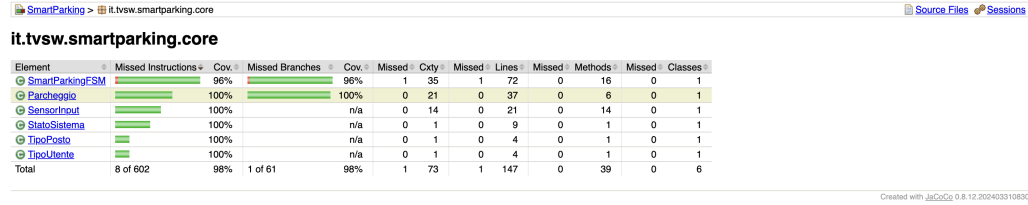


Figura 18: Report JaCoCo – dettaglio per classe del package core

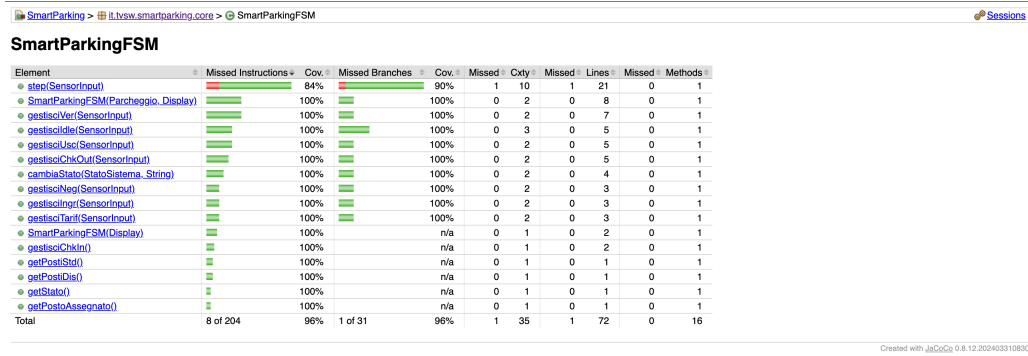


Figura 19: Report JaCoCo – dettaglio per metodo di SmartParkingFSM

4.3 Un criterio più forte: MC/DC

Il 98% di branch coverage appena raggiunto ha un limite che non si vede dal numero. La branch coverage chiede che ogni ramo di ogni `if` sia stato percorso almeno una volta, ma non dice niente su *quale* delle condizioni dentro quel test lo abbia fatto scattare. Una decisione come `(utente == STD || utente == ABBONATO) && postiStd > 0` risulta coperta con due soli casi di test, uno che la rende vera e uno falsa, anche se una delle tre condizioni non ha mai influenzato l'esito. Se quella condizione fosse scritta male, i test passerebbero lo stesso e il branch risulterebbe verde.

MC/DC serve esattamente a questo: chiede che per *ogni* condizione atomica esista una coppia di casi in cui cambia solo quella condizione e cambia anche il risultato della decisione. È il criterio che dimostra che ciascuna condizione, da sola, conta qualcosa.

Dentro `assegnaPosto` ci sono due decisioni composte, cioè con più di una condizione legata da and/or, ed è su quelle che l'ho applicato. Vediamole una alla volta.

La prima è quella del ramo `STD/ABBONATO`: $D_1 = (\text{utente}=\text{STD} \vee \text{utente}=\text{ABBONATO}) \wedge \text{postiStd}>0$. Le tre condizioni sono chiamate A, B e C nell'ordine in cui compaiono.

Caso	A	B	C	D_1	Esito	Coppia indipendente
TC1	T	F	T	T	POSTO_STD	base

Caso	A	B	C	D_1	Esito	Coppia indipendente
TC2	T	F	F	F	NESSUNO	isola C (vs TC1)
TC3	F	F	T	F	POSTO_DIS	isola A (vs TC1)
TC4	F	T	T	T	POSTO_STD	isola B (vs TC3)

Tabella 6: MC/DC – Decisione 1.

Leggendola: TC1 è il caso base, un utente STD con un posto libero. TC2 cambia solo C (niente posti) e la decisione diventa falsa, quindi la coppia TC1–TC2 isola C. TC3 cambia solo A rispetto a TC1 ed è la coppia per A. TC4 confrontato con TC3 cambia solo B ed è la coppia per B. Con quattro casi sono coperte tutte e tre le condizioni.

La seconda decisione è quella del ramo DISABILE, cioè il fallback, con $C_1 = \text{postiDis} > 0$ e $C_2 = \text{postiStd} > 0$.

Caso	C_1	C_2	Esito	Coppia indipendente
TC5	T	—	POSTO_DIS	base
TC6	F	T	POSTO_STD	isola C_1 (vs TC5)
TC7	F	F	NESSUNO	isola C_2 (vs TC6)

Tabella 7: MC/DC – Decisione 2.

Qui in TC5 il valore di C_2 non conta, perché se c'è il posto disabili il secondo if non viene nemmeno valutato: per questo c'è un trattino.

4.4 Combinatorial testing (pairwise)

L'assegnazione del posto dipende da tre parametri: il tipo di utente (tre valori) e i due contatori, che nel modello valgono 0 oppure 1. Tutte le combinazioni sarebbero $3 \times 2 \times 2 = 12$, che sono poche e si potrebbero anche fare tutte, ma il punto dell'esercizio era usare il pairwise. Basta quindi coprire tutte le *coppie* di valori, e per farlo bastano 6 righe invece di 12.

#	tipoUtente	postiStd	postiDis	Esito atteso
1	STD	0	0	NESSUNO
2	STD	1	1	POSTO_STD
3	DISABILE	0	1	POSTO_DIS
4	DISABILE	1	0	POSTO_STD (fall-back)
5	ABBONATO	0	1	NESSUNO
6	ABBONATO	1	0	POSTO_STD

Tabella 8: Tabella pairwise (copre tutte le coppie dei 3 parametri).

Le righe 3 e 5 sono le più interessanti, perché messe vicine fanno vedere la regola del posto riservato: stessa configurazione di contatori, ma il disabili entra e l'abbonato no.

4.5 Mockito

La FSM, ogni volta che cambia stato, manda un messaggio a un oggetto `Display`. Per controllare che questi messaggi partano davvero senza tirare in ballo la UI c'è Mockito: si crea un `Display` finto e poi con `verify` si controlla che il metodo `mostra` sia stato chiamato con il testo giusto.

Listing 7: Verifica delle notifiche con Mockito

```

1 @Test
2 void notificaAccessoNegatoQuandoParcheggioPieno() {
3     SmartParkingFSM fsm = new SmartParkingFSM(new Parcheggio(0, 0), display);
4
5     fsm.step(SensorInput.sensIn());
6     fsm.step(SensorInput.utente(TipoUtente.STD));
7     fsm.step(SensorInput.utente(TipoUtente.STD));
8
9     verify(display).mostra("Veicolo rilevato in ingresso");
10    verify(display).mostra("Verifica utente in corso");
11    verify(display).mostra("Accesso negato");
12    verifyNoMoreInteractions(display);
13 }

```

Il test parte da un parcheggio già pieno, fa arrivare un'auto e controlla l'intera sequenza di messaggi, non solo l'ultimo. La riga che conta di più è `verifyNoMoreInteractions`: senza di quella il test direbbe soltanto “fra le altre cose ha mostrato Accesso negato”, mentre così afferma che il display ha ricevuto esattamente quei tre messaggi e nient'altro. È la differenza fra verificare che qualcosa sia successo e verificare che sia successo *solo* quello.

4.6 Model Based Testing

Tutti i test visti finora sono nati guardando il codice. Il model based testing rovescia la direzione: il caso di test si ricava dal *modello*, e il codice diventa l'oggetto sotto esame invece che la fonte. In questo progetto il modello c'è già ed è validato, e gli scenari Avalla sono a tutti gli effetti dei casi di test scritti su di esso: portarli sul codice è quindi un passaggio quasi meccanico.

`test_pieno.avalla` è tradotto riga per riga in `ScenarioAvallaTest` seguendo tre corrispondenze fisse:

Avalla	JUnit
<code>set f := v;</code>	preparazione dell'input, tramite i metodi statici di <code>SensorInput</code>
<code>step</code>	una chiamata a <code>fsm.step(input)</code>
<code>check e = v;</code>	un <code>assertEquals</code> sul getter corrispondente

Listing 8: Estratto di `ScenarioAvallaTest`: ogni blocco cita la riga dell'avalla

```

1 // riga 9-11: set sens_in := true; step -> check stato = CHK_IN;
2 fsm.step(SensorInput.sensIn());
3 assertEquals(StatoSistema.CHK_IN, fsm.getStato());
4
5 // riga 40-41: step (posti_std = 0 -> nega accesso) -> check stato = NEG;
6 fsm.step(SensorInput.utente(TipoUtente.STD));
7 assertEquals(StatoSistema.NEG, fsm.getStato());

```

Ogni blocco porta in commento la riga originale dello scenario, così il confronto fra i due file si fa senza tenerli aperti tutti e due. Non è pignoleria: è ciò che rende il test *tracciabile* al modello, e quindi rifacibile se il modello cambia.

Una differenza fra i due livelli merita attenzione, perché è il punto in cui la traduzione non è del tutto meccanica. Nell'ASM ogni `set` imposta una funzione monitorata che resta impostata finché non la cambio, mentre in Java ogni `SensorInput` è un oggetto immutabile che descrive un singolo step. Dove l'avalla scrive `set sens_in := false` per spegnere un sensore, il test

Java semplicemente costruisce l'input successivo senza quel sensore: il risultato è lo stesso, ma la corrispondenza è uno-a-uno solo sugli **step** e sui **check**, non sui **set**.

Il valore della cosa si è visto sul branch `ci_failure` (Sezione 7): quando `assegnaPosto` è stata rotta di proposito, questo test è fra quelli che sono andati in rosso — cioè una regressione nel codice è stata intercettata da un caso di test che nessuno aveva scritto guardando il codice.

4.7 Mutation testing con PIT

Statement, branch e MC/DC hanno tutti la stessa cieca in comune, ed è l'ultimo gradino della scala. Dicono che una riga è stata *eseguita*, o che una condizione ha *influenzato* una decisione, ma nessuno dei tre guarda cosa succede dopo: se il risultato di quell'esecuzione sia stato *controllato* da qualcuno. Un test che chiama un metodo e non verifica niente alza la copertura, MC/DC compreso, esattamente come un test scritto bene.

Il mutation testing serve proprio a distinguere i due casi. PIT prende il bytecode e ci introduce piccoli difetti uno alla volta: cambia un `>` in `>=`, nega una condizione, sostituisce un valore di ritorno, cancella una chiamata a un metodo `void`. Per ogni difetto rilancia i test. Se i test falliscono il mutante è *ucciso*, e vuol dire che quella riga è sorvegliata da un assert. Se invece passano lo stesso il mutante *sopravvive*, e ho trovato un punto cieco.

Il plugin è configurato nel `pom.xml` sul solo package `core`, e si lancia così:

Listing 9: Esecuzione di PIT

```
$ mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

Il risultato è che su **90 mutanti generati ne sono stati uccisi 90**, cioè un mutation score del 100%, con zero mutanti privi di copertura.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	99% 127/128	100% 90/90	100% 90/90

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
it.tvsw.smartparking.core	3	99% 127/128	100% 90/90	100% 90/90

Report generated by [PIT](#) 1.15.8

Enhanced functionality available at [arcmutate.com](#)

Figura 20: Report PIT – riepilogo sul package core

Nel riepilogo si vedono due numeri diversi che è bene non confondere. La *line coverage* è 127 righe su 128, e quella riga scoperta è sempre il `default` irraggiungibile dello switch di cui sopra. La *mutation coverage* è 90 su 90, e dice tutt'altro: non quante righe sono state eseguite, ma su quante di esse un difetto introdotto di proposito è stato intercettato dai test.

Operatore di mutazione	Generati	Uccisi	%
NegateConditionals	26	26	100
VoidMethodCall	21	21	100
NullReturnVals	15	15	100

Operatore di mutazione	Generati	Uccisi	%
ConditionalsBoundary	9	9	100
BooleanTrueReturnVals	5	5	100
BooleanFalseReturnVals	5	5	100
Math	5	5	100
PrimitiveReturns	4	4	100
Totale	90	90	100

Tabella 10: Mutanti generati da PIT sul package `core`, per operatore.

Un mutante da vicino

Per capire cosa vuol dire in concreto “mutante ucciso” conviene guardarne uno solo. Prendiamo il ramo STD/ABBONATO di `assegnaPosto`, dove il codice originale è questo:

Listing 10: Codice originale

```

1 if (postiStd > 0) {
2     postiStd = postiStd - 1;
3     return TipoPosto.POSTO_STD;
4 }
5 return TipoPosto.NESSUNO;

```

L’operatore `ConditionalsBoundary` sposta il limite del confronto, trasformando `postiStd > 0` in `postiStd >= 0`. È una modifica di un carattere, e in tutte le esecuzioni con almeno un posto libero non cambia assolutamente niente: il codice mutato si comporta come l’originale.

La differenza salta fuori in un caso solo, quello con `postiStd == 0`. Lì l’originale salta il blocco e restituisce `NESSUNO`, mentre il mutante entra, decrementa il contatore portandolo a `-1` e restituisce `POSTO_STD`, cioè fa entrare un’auto in un parcheggio pieno.

Quel mutante viene ucciso dal primo caso della tabella pairwise (Tabella 8), l’unico che prova un utente STD con entrambi i contatori a zero. Ed è interessante notare che lo stesso difetto sarebbe stato intercettato anche dagli altri due livelli di verifica, ognuno a modo suo: il contatore a `-1` viola l’invariante `JML 0 <= postiStd` (Sezione 5) e viola la CTLSPEC `ag(posti_std >= 0)` verificata sul modello (§2.3). Tre tecniche diverse che convergono sullo stesso stato illegale.

Due righe della tabella valgono più delle altre, perché ricollegano il risultato al lavoro fatto prima. I 9 mutanti `ConditionalsBoundary` trasformano `postiStd > 0` in `postiStd >= 0`: ucciderli tutti vuol dire che i test distinguono davvero “c’è un posto” da “non ce n’è”, ed è precisamente ciò che i casi di confine della tabella MC/DC servivano a costruire. I 21 `VoidMethodCall` sono invece le chiamate a `display.mostra(...)` cancellate: non cambiano lo stato della FSM, quindi un test che guarda solo lo stato finale non se ne accorgerebbe mai, e a ucciderli sono i `verify` di Mockito.

Quanto vale davvero questo 100%

Il punteggio pieno va letto per quello che è. Il nucleo è piccolo e fatto apposta per essere verificabile — niente cicli, niente strutture dati, aritmetica di soli incrementi — quindi i mutanti possibili sono pochi e coprirli tutti è realistico. E soprattutto PIT ha usato il suo insieme di operatori predefinito: attivandone di più aggressivi qualcuno probabilmente sopravviverebbe. Il 100% quindi non significa “nessun difetto sfugge ai miei test” ma “nessun difetto *di questa famiglia* sfugge ai miei test” — la stessa distinzione che c’è fra copertura e mutation testing, spostata di un livello. Quello che il risultato dimostra sul serio è ciò che cercavo: il 98% di copertura non è gonfiato, perché le righe eseguite sono anche righe i cui effetti vengono controllati.

5 JML e verifica statica (ESC)

Per la parte di design by contract è annotata con JML una classe sola, **Parcheggio**. È il nucleo del dominio: tiene i due contatori dei posti liberi e gestisce l'assegnazione e il rilascio, cioè fa quello che nel modello ASM fanno le regole **r_gestione_VER**, **r_gestione_INGR** e **r_gestione_USC**.

È piccola apposta e senza dipendenze da niente: dentro ci sono solo **int** ed **enum**. Il motivo è pratico, cioè che così OpenJML riesce a dimostrarla tutta, mentre su una classe che si tira dietro mezzo Spring non ci sarei mai arrivato. Tutto il resto del sistema (la FSM, la UI, il display, i sensori) è Java normale senza annotazioni, come chiedeva la consegna.

5.1 Invarianti

Le invarianti sono due e devono valere sempre, da dopo il costruttore in poi:

- i posti standard liberi stanno sempre tra 0 e **MAX_STD**;
- i posti disabili liberi stanno sempre tra 0 e **MAX_DIS**.

Listing 11: Invarianti di classe

```
//@ public invariant 0 <= postiStd && postiStd <= MAX_STD;
private /*@ spec_public @*/ int postiStd;

//@ public invariant 0 <= postiDis && postiDis <= MAX_DIS;
private /*@ spec_public @*/ int postiDis;
```

Sono le stesse identiche proprietà già verificate con il model checking, cioè **ag(posti_std >= 0)** e **ag(posti_std <= 1)** con le corrispondenti sui posti disabili. Le ho scritte apposta uguali, così la stessa proprietà finisce controllata a tre livelli: sul modello astratto dal model checker, che la dimostra su tutti gli stati raggiungibili; sul codice dal contratto JML, che la dimostra su tutte le esecuzioni possibili; e a runtime dagli assert di **ParcheggioTest**, che la controllano sui casi effettivamente eseguiti. Sono tre garanzie di forza decrescente ma su oggetti diversi, ed è il motivo per cui ha senso averle tutte e tre.

5.2 Costruttori

I costruttori sono due. Quello di default garantisce lo stato iniziale del modello ASM, cioè tutti i posti liberi. Quello con i parametri serve nei test, per partire da una configurazione qualunque, e chiede che i valori passati siano già dentro i limiti dell'invariante.

Listing 12: Contratti dei costruttori

```
//@ ensures postiStd == MAX_STD && postiDis == MAX_DIS;
public Parcheggio() { ... }

//@ requires 0 <= postiStdIniziali && postiStdIniziali <= MAX_STD;
//@ requires 0 <= postiDisIniziali && postiDisIniziali <= MAX_DIS;
//@ ensures postiStd == postiStdIniziali && postiDis == postiDisIniziali;
public Parcheggio(int postiStdIniziali, int postiDisIniziali) { ... }
```

Il **requires** del secondo costruttore dice cosa deve garantire chi chiama, ma non protegge a runtime: se qualcuno passa un valore fuori range senza aver fatto la verifica statica, il contratto non lo ferma. Per questo dentro c'è anche un controllo esplicito che lancia **IllegalArgumentException**, provato in **ParcheggioTest** con **assertThrows**.

5.3 Metodo assegnaPosto

Questo è quello con il contratto più lungo, perché deve dire tutto quello che fa `r_gestione_VER`. Le postcondizioni sono due, una per ramo.

Per STD e ABBONATO ci sono due esiti possibili: o c'era un posto standard libero, e allora torna `POSTO_STD` con il contatore sceso di 1, oppure non c'era e torna `NESSUNO` senza toccare niente.

Per il DISABILE gli esiti sono tre, perché in mezzo c'è il fallback: posto disabile se disponibile, altrimenti posto standard se disponibile, altrimenti `NESSUNO`.

Listing 13: Contratto di assegnaPosto

```
//@ requires utente != null;
//@ ensures (utente == TipoUtente.STD || utente == TipoUtente.ABBONATO) ==>
//@      ((\old(postiStd) > 0 && \result == TipoPosto.POSTO_STD
//@      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
//@      || (\old(postiStd) == 0 && \result == TipoPosto.NESSUNO
//@      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
//@ ensures utente == TipoUtente.DISABILE ==>
//@      ((\old(postiDis) > 0 && \result == TipoPosto.POSTO_DIS
//@      && postiDis == \old(postiDis) - 1 && postiStd == \old(postiStd))
//@      || (\old(postiDis) == 0 && \old(postiStd) > 0
//@      && \result == TipoPosto.POSTO_STD
//@      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
//@      || (\old(postiDis) == 0 && \old(postiStd) == 0
//@      && \result == TipoPosto.NESSUNO
//@      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
public TipoPosto assegnaPosto(TipoUtente utente) { ... }
```

Il `\old` serve per parlare del valore che il contatore aveva prima della chiamata, altrimenti non si potrebbe dire “è sceso di uno”.

Una cosa che all'inizio mancava: in ogni esito è scritto anche che l'*altro* contatore resta com'era. Senza quel pezzo il contratto sarebbe sottospecificato, cioè un'implementazione che assegna il posto giusto ma nel frattempo tocca anche l'altro contatore lo rispetterebbe lo stesso.

5.4 Metodo liberaPosto

Questo corrisponde al rilascio del posto in `r_gestione_USC`. Chiede che il posto non sia null e che il contatore corrispondente non sia già al massimo, e garantisce che venga incrementato di 1 solo quello giusto. Con `NESSUNO` non fa niente e lo stato resta identico.

Listing 14: Contratto di liberaPosto

```
//@ requires posto != null;
//@ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
//@ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
//@ ensures posto == TipoPosto.POSTO_STD ==>
//@      postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
//@ ensures posto == TipoPosto.POSTO_DIS ==>
//@      postiDis == \old(postiDis) + 1 && postiStd == \old(postiStd);
//@ ensures posto == TipoPosto.NESSUNO ==>
//@      postiStd == \old(postiStd) && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

Quel secondo e terzo `requires` non c'erano nella prima versione e sono il motivo per cui la prova non chiudeva: ne parlo tra poco.

5.5 Metodi getter

I due getter sono annotati `/*@ pure @*/`, che vuol dire senza effetti collaterali. Serve perché un metodo non puro dentro una clausola JML non si può usare. La postcondizione dice solo che restituiscono il campo, niente di più.

Listing 15: Getter puri

```
//@ ensures \result == postiStd;
public /*@ pure @*/ int getPostiStd() { ... }

//@ ensures \result == postiDis;
public /*@ pure @*/ int getPostiDis() { ... }
```

5.6 Verifica dei contratti con OpenJML (ESC)

La verifica dei contratti è stata fatta con il plugin OpenJML per Eclipse (build 0.8.59-20211116, quella distribuita nel toolkit del corso), lanciando il comando *Static Checks (ESC)* sulla classe `Parcheggio` e usando `z3_4_3` come solver. Quello che fa il tool è tradurre codice e contratti in formule logiche e passarle a Z3, che deve dimostrare che *qualunque* esecuzione rispetta postcondizioni e invarianti. È una cosa diversa dai test: i test provano alcuni casi, qui invece o si dimostra per tutti o non si dimostra.

L'esito compare in due posti. Nella vista *Static Checks* ogni metodo è marcato [VALID] o [INVALID], con accanto il tempo di prova e il solver usato; nella *JML Console* scorre il dettaglio, un *feasibility check* alla volta, e in fondo il riepilogo con il conteggio per categoria.

5.6.1 Prima esecuzione: cinque metodi su sei

Al primo lancio la verifica non chiude. Cinque metodi risultano VALID, ma `liberaPosto` è marcato [INVALID] in rosso, e sotto di esso compaiono due nodi `CE#1` e `CE#2`: sono i due controesempi che il solver ha costruito per dimostrare che il contratto, così com'è scritto, non regge. Il riepilogo dice `Valid: 5 / Invalid: 1` e, soprattutto, `Classes: 0 proved of 1`: basta un metodo non dimostrato perché l'intera classe risulti non verificata.

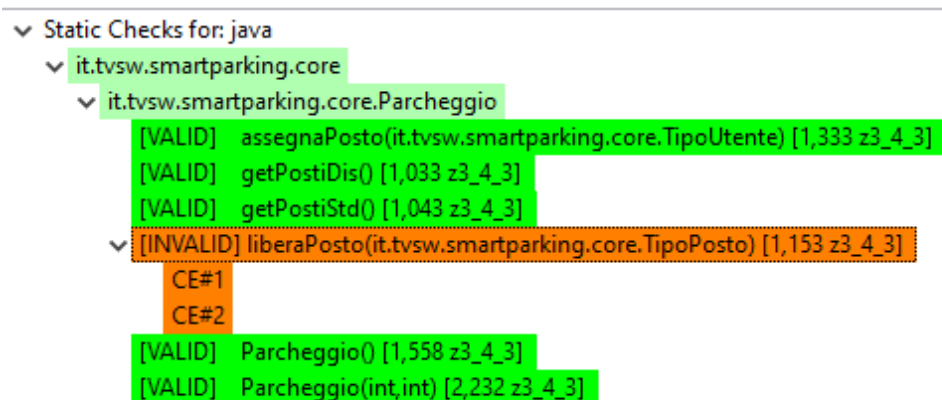


Figura 21: Prima esecuzione: `liberaPosto` non dimostrabile

La versione sotto esame era questa: la postcondizione descrive l'incremento, ma niente dice al solver da quale valore si parte.

Listing 16: Prima versione di liberaPosto (senza il requires di capacità)

```

/*@ requires posto != null;
  @ ensures posto == TipoPosto.POSTO_STD ==>
    postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
  @ ensures posto == TipoPosto.POSTO_DIS ==>
    postiDis == \old(postiDis) + 1 && postiStd == \old(postiStd);
public void liberaPosto(TipoPosto posto) {
    if (posto == TipoPosto.POSTO_STD) {
        postiStd = postiStd + 1;
    } else if (posto == TipoPosto.POSTO_DIS) {
        postiDis = postiDis + 1;
    }
}

```

5.6.2 Perché è invalido: lettura del controesempio

Il vantaggio dell'ESC rispetto a un test che fallisce è che qui non si ottiene solo un “no”: si ottiene l'esecuzione precisa che rompe il contratto. La traccia completa è lunga qualche centinaio di righe, ma la maggior parte riguarda le strutture interne dell'enum `TipoPosto` (i vari `_JMLvalues`, `charArray`) e non dice niente di interessante. La parte che conta sono le ultime righe di **CE#1** (i numeri di riga si riferiscono alla versione con il contratto incompleto, non a quella definitiva):

Listing 17: Estratto di CE#1: il ramo del posto disabile

```

Parcheggio.java:112: requires posto != null;
  VALUE: posto != null === true
Parcheggio.java:  if (posto == TipoPosto.POSTO_STD) ...
  Condition = false
Parcheggio.java:  if (posto == TipoPosto.POSTO_DIS) ...
  Condition = true
Parcheggio.java:116: postiDis = postiDis + 1
  VALUE: postiDis === 1
  VALUE: postiDis + 1 === 2
Parcheggio.java:35:  //@ public invariant 0 <= postiDis && postiDis <= MAX_DIS;
  VALUE: postiDis === 2
  VALUE: MAX_DIS === 1
  VALUE: postiDis <= MAX_DIS === false
Parcheggio.java:112: Invalid assertion (InvariantExit)
      : Parcheggio.java:35: Associated location

```

Si legge come una sceneggiatura. La preconditione è soddisfatta (`posto != null`), l'esecuzione entra nel ramo del posto disabile, il contatore parte da 1 e diventa 2. A quel punto il tool ricontra l'invariante di classe e trova $2 \leq 1$ falso: l'assertion che salta si chiama `InvariantExit`, cioè “invariante all'uscita del metodo”, e la *associated location* indica la riga 35, dove l'invariante è dichiarata.

Il punto è che il solver non ha nessun motivo per escludere il caso `postiDis == MAX_DIS`. Nessuno glielo ha vietato, quindi lo prova, e trova la crepa. **CE#2** è la stessa identica storia sull'altro ramo, con `postiStd` che passa da 1 a 2: due controesempi perché i rami con un incremento sono due.

5.6.3 La correzione e la seconda esecuzione

Il codice non è sbagliato, è il contratto a essere incompleto: manca l'ipotesi che il posto da liberare fosse davvero occupato. La si aggiunge come preconditione, spostando l'onere su chi chiama:

Listing 18: Versione corretta: il requires di capacità chiude la prova


```
//@ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
//@ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
```

Con `postiDis < MAX_DIS` assunto all'ingresso, dopo l'incremento vale `postiDis <= MAX_DIS`: l'esecuzione di `CE#1` non è più ammessa e l'invariante regge. Nel codice ho aggiunto anche una guardia esplicita che solleva `IllegalStateException`, perché la preconditione vincola solo chi rispetta il contratto, mentre a runtime un chiamante distratto deve comunque essere fermato.

Rilanciando l'ESC, tutti e sei i metodi (i due costruttori, `assegnaPosto`, `liberaPosto` e i due getter) passano a `VALID`.

```
▼ Static Checks for: java
  ▼ it.tvsw.smartparking.core
    ▼ it.tvsw.smartparking.core.Parcheggio
      [VALID] assegnaPosto(it.tvsw.smartparking.core.TipoUte) [1,366 z3_4_3]
      [VALID] getPostiDis() [1,040 z3_4_3]
      [VALID] getPostiStd() [1,042 z3_4_3]
      [VALID] liberaPosto(it.tvsw.smartparking.core.TipoPosto) [1,464 z3_4_3]
      [VALID] Parcheggio() [1,241 z3_4_3]
      [VALID] Parcheggio(int,int) [2,374 z3_4_3]
```

Figura 22: Seconda esecuzione: tutti i metodi `VALID`

Il riepilogo della JML Console lo conferma: `Valid: 6`, con `Invalid`, `Infeasible`, `Timeout`, `Error` e `Skipped` tutti a zero, e la classe che passa da 0 `proved of 1` a 1 `proved of 1`.

```
JML Console
C:\Users\Administrator\Desktop\WKS\java\src\main\java\it\tvsw\smartparking\core\Parcheggio.java:138: Feasibility check #1 - end of preconditions : OK [0,00 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUte) with prover z3_4_3 - no warnings [1,80 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3_4_3
Method assertions are validated [1,04 secs]
C:\Users\Administrator\Desktop\WKS\java\src\main\java\it\tvsw\smartparking\core\Parcheggio.java:138: Feasibility check #2 - at program exit : OK [0,00 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3_4_3 - no warnings [1,07 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3_4_3
Method assertions are validated [1,04 secs]
C:\Users\Administrator\Desktop\WKS\java\src\main\java\it\tvsw\smartparking\core\Parcheggio.java:133: Feasibility check #1 - end of preconditions : OK [0,00 secs]
C:\Users\Administrator\Desktop\WKS\java\src\main\java\it\tvsw\smartparking\core\Parcheggio.java:133: Feasibility check #2 - at program exit : OK [0,00 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3_4_3 - no warnings [1,07 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3_4_3
Method assertions are validated [1,43 secs]
C:\Users\Administrator\Desktop\WKS\java\src\main\java\it\tvsw\smartparking\core\Parcheggio.java:114: Feasibility check #26 - end of preconditions : OK [0,42 secs]
C:\Users\Administrator\Desktop\WKS\java\src\main\java\it\tvsw\smartparking\core\Parcheggio.java:114: Feasibility check #153 - at program exit : OK [0,05 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3_4_3 - no warnings [2,02 secs]
Completed proving methods in it.tvsw.smartparking.core.Parcheggio [12,72 secs]
Summary:
Valid: 6
Invalid: 0
Infeasible: 0
Timeout: 0
Error: 0
Skipped: 0
TOTAL METHODS: 6
Classes: 1 proved of 1
Model Classes: 0
Model methods: 0 proved of 0
DURATION: 13,4 secs
[14,66] Completed
[0,00] Executing openjml on Parcheggio.java
RAC-Compiling it.tvsw.smartparking.core.Parcheggio
RAC-Compiling it.tvsw.smartparking.core.TipoUte
RAC-Compiling it.tvsw.smartparking.core.TipoPosto
[1,35] Completed
```

Figura 23: JML Console: riepilogo della verifica

5.7 Verifica dinamica: i contratti a runtime (RAC)

Vedere sei righe verdi “`VALID`” è comodo, ma da solo non mi diceva se i contratti stessero davvero controllando qualcosa o se fossero solo scritti bene. È lo stesso dubbio che mi era venuto con la pipeline verde della CI: passa, ma se rompo qualcosa se ne accorge?

Per togliermelo ho usato l'altra modalità del plugin, il *RAC* (Runtime Assertion Checking), che si lancia dallo stesso menu dell'ESC. Qui il tool non dimostra niente in anticipo: ricompila le

classi infilando i controlli dentro il bytecode – nella JML Console si vedono scorrere le righe **RAC-Compiling** per **Parcheggio**, **TipoUtente** e **TipoPosto** – e i contratti vengono poi verificati mentre il programma gira. Se una clausola salta, l’esecuzione si ferma dicendo quale.

Il programma di prova è **DemoRac** (package `it.tvsw.smartparking.jml`), che serve solo a questo e non viene chiamato né dalla FSM né dalla UI. Fa tre cose in fila: prima un uso legittimo, che non deve dare nessun errore, e poi due violazioni fatte apposta.

Listing 19: Le due violazioni volontarie in **DemoRac**

```

1 // 1. Viola il requires del costruttore: MAX_STD vale 1, qui ne passo 5
2 Parcheggio p = new Parcheggio(5, 0);
3
4 // 2. Viola il requires di liberaPosto: il contatore e' gia' al massimo,
5 //   perche' non e' entrato nessuno, quindi l'incremento sfonderebbe l'
6 //   invariante
7 Parcheggio q = new Parcheggio();
8 q.liberaPosto(TipoPosto.POSTO_STD);

```

Il procedimento è in due passi: prima si lancia il RAC sulla classe da strumentare, e il plugin ricompila **Parcheggio** con i controlli dentro; poi si esegue **DemoRac** come una normale applicazione Java, avendo cura che il classpath punti alle classi strumentate e includa il runtime JML del plugin (`jmlruntime.jar`) invece dei `.class` prodotti dal compilatore di Eclipse.

La seconda violazione è quella che mi interessava di più, perché è esattamente il caso che aveva fatto fallire la prova ESC prima che aggiungessi il **requires** sulla capacità. Prima quel caso era un buco nella specifica e il solver me lo segnalava come **Invalid assertion (InvariantExit)**; adesso che il **requires** c’è, lo stesso caso diventa una precondizione violata dal chiamante, e a runtime si vede subito.

Le due modalità guardano lo stesso identico punto da due lati, ed è questa la ragione per cui ho voluto provarle entrambe. L’ESC ragiona su *tutte* le esecuzioni possibili e quando fallisce dice “non riesco a dimostrare che questa proprietà valga sempre”, consegnandoti un controesempio costruito dal solver. Il RAC non dimostra nulla, ma sull’esecuzione che gli dai in pasto ti dice “ecco, qui adesso non vale”. Il primo è esaustivo ma astratto, il secondo è concreto ma limitato a ciò che esegui davvero: presi insieme confermano che i contratti non sono decorativi, perché lo stesso vincolo che il solver usa per chiudere la prova è quello che a runtime ferma il programma.

6 Ispezione del codice e analisi statica

Il codice è passato sotto tre filtri diversi: una rilettura critica fatta fare a un LLM, la checklist di code inspection vista a lezione e l’analisi automatica con SpotBugs. Il primo è quello che ha reso di più ed è quello a cui dedico la maggior parte della sezione; gli altri due li riassumo dopo, insieme a cosa ha trovato ciascuno che gli altri non potevano vedere.

6.1 Rilettura critica con un LLM

Ho usato Claude Opus 4.7 come se fosse un collega a cui far dare una seconda occhiata al codice. Gli ho passato tutto il sorgente dei package `core` e `ui` e gli ho chiesto di leggerlo cercando nomi poco chiari, casi limite non gestiti, possibili `NullPointerException` e punti in cui il codice si allontana dal modello ASM.

Le cose che sono uscite non le ho prese per buone così com’erano: le ho valutate una per una, decidendo di volta in volta se correggerle, mitigarle o lasciarle stare spiegando perché. L’LLM ha segnalato i punti su cui ragionare, le decisioni le ho prese io.

1. **TipoPosto.NESSUNO vuol dire due cose diverse:** in `assegnaPosto` significa “non c’è posto”, in `liberaPosto` significa “non fare niente”. Chiarito nel Javadoc, invece di inventare un tipo separato che avrebbe appesantito una classe che doveva restare semplice.
2. **`liberaPosto` lancia eccezione se il contatore è già al massimo.** Questa l’ho tenuta apposta: preferisco un crash che si vede subito a un contatore che va oltre la capacità senza dire niente, violando l’invariante. Il caso è comunque coperto da un test.
3. **Il decremento avviene in un momento diverso rispetto all’ASM:** qui `assegnaPosto` decide e scala insieme, nell’ASM si scala al transito. Negli scenari di test la differenza non si vede, quindi resta la versione più semplice, scritta nel Javadoc della classe.
4. **I bottoni della UI sono sempre attivi:** `MainView` non li disabilita in base allo stato, quindi si può cliccare “Pagamento” mentre siamo in IDLE. Non succede niente di male perché quegli step finiscono assorbiti dai self-loop della FSM, cioè proprio dagli `if` senza `else` che il Model Advisor segnalava come MP2 (§2.4.1). Per un prototipo va bene; in una versione vera si metterebbe un `setEnabled` legato allo stato.
5. **`posto_assegnato` è una memoria da un posto solo, e il difetto c’è anche nel modello ASM.** Questa è l’osservazione più importante di tutte. Sia la FSM Java sia l’ASM si ricordano un solo posto assegnato, ma nel parcheggio ci stanno due veicoli, uno standard e uno disabili. Se entra uno STD, poi entra un DISABILE che sovrascrive lo slot, e poi esce il primo, viene rilasciato il posto sbagliato e i contatori si sballano.

Il punto è che l’implementazione è fedele alla specifica: è la specifica stessa che dà per scontato di tracciare un veicolo alla volta, senza mai associare un veicolo al suo posto. Per sistemarlo servirebbe una mappa veicolo→posto sia nell’ASM sia in Java. L’ho lasciato e documentato come limite noto, perché mi sembra l’esempio più utile di tutto il progetto: è un difetto che il testing di conformità non può trovare, visto che il codice fa esattamente quello che dice il modello, e che salta fuori solo leggendo il codice con occhio critico.

Quest’ultimo punto è anche quello che dà il senso all’intera sezione. Né il testing di conformità né la verifica ESC potevano vederlo: i contratti JML di `Parcheggio` restano rispettati anche nell’esecuzione che rilascia il posto sbagliato, perché il problema sta più in alto, a livello di FSM e di modello. Il model checking e l’ESC dimostrano quello che è stato formalizzato; l’ispezione trova quello che nella formalizzazione non c’è.

6.2 Gli altri due filtri, in breve

La **checklist** è meccanica e va per categorie, quindi mi ha costretto a guardare le *firme* dei metodi, cosa che leggendo il codice per capire cosa fa non si fa mai. Da lì è saltata fuori una segnalazione che nessuno degli altri due filtri aveva visto: sia `Parcheggio(int, int)` sia il costruttore di `SensorInput` hanno **parametri adiacenti dello stesso tipo**, quindi uno scambio in chiamata compila senza un fiato e cambia il comportamento. Su `SensorInput` il rischio non si presenta perché il costruttore lo si usa sempre tramite i metodi statici di comodo; su `Parcheggio` resta, ma quel costruttore lo chiamano solo i test.

SpotBugs (`effort=Max, threshold=Low`, lanciato con `mvn spotbugs:check`) ha prodotto 7 segnalazioni, divise in due gruppi. Quattro `CT_CONSTRUCTOR_THROW` sui costruttori che validano i parametri: i `throw` sono però proprio i controlli difensivi che fanno rispettare a runtime i **requires** JML, quindi invece di toglierli ho dichiarato `final` le classi `Parcheggio` e `SmartParkingFSM`, che è ciò che rende impossibile il *finalizer attack* presupposto dalla segnalazione. Gli altri tre riguardano la serializzazione ed è il gruppo che vale: `MainView` teneva un campo non serializzabile, e siccome Vaadin mette l’intero albero dei componenti nella sessione HTTP, con la replicazione di sessione l’applicazione si sarebbe piantata. È un difetto vero ma

latente, che né la checklist né l'LLM potevano vedere perché richiede di conoscere i contratti interni di Vaadin. Dopo le correzioni (classi serializzabili, `Display` che estende `Serializable` per via della lambda, e `serialVersionUID` ovunque) SpotBugs riporta `BugInstance size is 0`, con i test tutti verdi e la copertura invariata.

7 Continuous Integration

La continuous integration gira su GitHub Actions. Il senso è che a ogni push o pull request i test partono da soli, così se rompo qualcosa me ne accorgo subito invece che due settimane dopo.

Tutto sta nel file `.github/workflows/ci.yml`: fa la build Maven completa, lascia fuori i test taggati `ui` (che hanno bisogno di Chrome e dell'app avviata, e sul runner non ci sono) e alla fine salva il report JaCoCo come artifact scaricabile.

Listing 20: `.github/workflows/ci.yml` (estratto)

```
1 on:
2   push:
3   pull_request:
4
5 jobs:
6   build-and-test:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - uses: actions/setup-java@v4
11        with:
12          distribution: temurin
13          java-version: '17'
14      - run: mvn -B -f java/pom.xml verify
15      - uses: actions/upload-artifact@v4
16        with:
17          name: jacoco-report
18          path: java/target/site/jacoco/
```

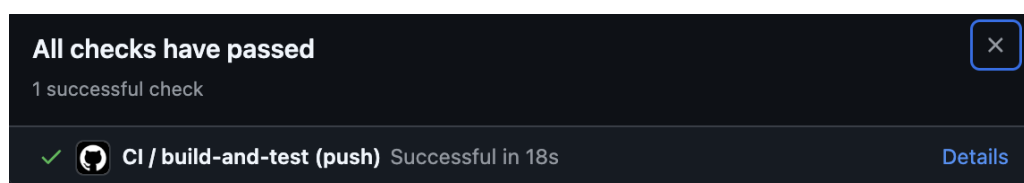


Figura 24: Esecuzione CI riuscita sul branch `main`

Questo è il run verde su `main`.

Solo che avere la pipeline verde, di per sé, dimostra poco: dimostra che i test passano, non che la CI se ne accorgerebbe se smettessero di passare. Per essere sicuro che il meccanismo funzionasse ho quindi rotto il codice apposta. Su un branch a parte, chiamato `ci_failure` e lasciato sul repository come prova, `assegnaPosto` è stato modificato per restituire sempre `TipoPosto.NESSUNO`. Al push la pipeline è andata in rosso, con i test di `ParcheggioTest` e `SmartParkingFSMTest` falliti sugli assert dell'assegnazione. Il branch `main` è rimasto com'era.

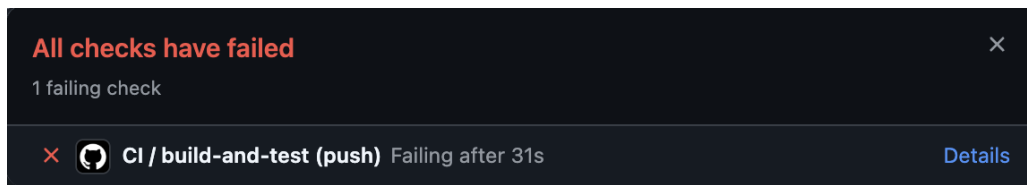


Figura 25: Esecuzione CI fallita sul branch `ci_failure` (regressione introdotta di proposito)

Quello che mi porto a casa da questa parte è che il valore della CI non è tanto far girare i test, che posso farlo anche in locale, quanto non potermene dimenticare: ogni commit resta legato a una prova che in quel momento il codice funzionava.